

agentic-ai-azure-week01-foundations

Week 1 - Foundations of Agentic AI

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week01-foundations' - Week 1 - Foundations of Agentic AI. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Understand the agent loop: reason -> plan -> act -> observe
- Learn tool use and the 'agent contract'
- Know where each Azure service fits

Notes

[Learning goal] Walk through these talking points:

- Understand the agent loop: reason -> plan -> act -> observe
- Learn tool use and the 'agent contract'
- Know where each Azure service fits

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- IT Helpdesk Triage Agent (cross-industry)
- Reads a ticket, assesses severity, answers from KB or escalates
- Establishes the mental model before adding complexity

Notes

[Enterprise use case] Walk through these talking points:

- IT Helpdesk Triage Agent (cross-industry)
- Reads a ticket, assesses severity, answers from KB or escalates
- Establishes the mental model before adding complexity

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Input: support message
- REASON: assess severity -> PLAN: answer vs escalate
- ACT: KB lookup tool -> OBSERVE: finalize
- Guardrail: urgent tickets escalate to a human

Notes

[Architecture / flow] Walk through these talking points:

- Input: support message
- REASON: assess severity -> PLAN: answer vs escalate
- ACT: KB lookup tool -> OBSERVE: finalize
- Guardrail: urgent tickets escalate to a human

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- One tool (KB lookup) + an escalation guardrail
- Mock backend = deterministic loop; Foundry backend = model classify
- Typed Pydantic request/response contract

Notes

[Key concepts] Walk through these talking points:

- One tool (KB lookup) + an escalation guardrail
- Mock backend = deterministic loop; Foundry backend = model classify
- Typed Pydantic request/response contract

Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/triage
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/triage
- Runs in MOCK mode - no Azure/keys needed

Pause for questions before moving on.

Slide 7 - Architect's takeaways

- When is an agent right vs. a workflow or single call?
- Define inputs, tools, guardrails, success criteria up front
- Foundry vs. Agent Framework vs. raw model calls

Notes

[Architect's takeaways] Walk through these talking points:

- When is an agent right vs. a workflow or single call?
- Define inputs, tools, guardrails, success criteria up front
- Foundry vs. Agent Framework vs. raw model calls

Discussion: when would you NOT choose this approach?